IsPilot API

This service exposes IsPilot through a REST API for enterprise channels such as Copilot Studio, Microsoft Teams, and custom applications.

Production Status: ☑ Live at https://ispilot-api-46y2f3tyja-uc.a.run.app

Architecture

The API communicates with Google Vertex AI Reasoning Engine (ID: 5375474415045705728) to orchestrate multi-agent conversations. Sessions are managed through Cloud Firestore with automatic fallback to in-memory storage.

```
Client Request
    ↓
  Cloud Run (Workload Identity)
    ↓
  FastAPI Application
    ↓
  Vertex AI Reasoning Engine
    ↓
  Agent Orchestration
    ↓
  BigQuery Analytics
```

Quick Start: Local Development

Setup

```
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# Set environment variables
export GOOGLE_CLOUD_PROJECT="corp-stro-salesinventory-prod"
export GOOGLE_CLOUD_LOCATION="us-central1"
export VERTEX_PROJECT_ID="corp-stro-salesinventory-prod"
export VERTEX_LOCATION="us-central1"
export VERTEX_ENGINE_ID="5375474415045705728"
export FIRESTORE_COLLECTION="user_sessions"
export SESSION_TIMEOUT_HOURS="8"

# For local testing with API key
export ISPILOT_API_KEY="your-test-api-key"
```

```
# Run the server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8080
```

Note: Application Default Credentials (ADC) will automatically detect and use available credentials. In local development, ensure you're authenticated:

```
gcloud auth application-default login
```

Using run.sh

```
chmod +x run.sh
./run.sh
```

Authentication

Cloud Run (Production)

Uses **Workload Identity** - no credential files required. The service account (`sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com`) is automatically bound to the Cloud Run service.

Client Authentication (calling the API):

```
# Get an identity token
AUTH_TOKEN=$(gcloud auth print-identity-token \
  --audiences https://ispilot-api-46y2f3tyja-uc.a.run.app)

# Use the token in requests
curl -X POST https://ispilot-api-46y2f3tyja-uc.a.run.app/chat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $AUTH_TOKEN" \
  -d '{
    "user_id": "sebastian",
    "message": "How is Talca Colin performing?"
  }'
```

Local Development

Use API Key authentication for testing:

```
curl -X POST http://localhost:8080/chat \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your-test-api-key" \
  -d '{
```

```
    "user_id": "sebastian",
    "message": "How is Talca Colin performing?"
  }'
```

Environment Variables

Variable	Required	Description	Default
`GOOGLE_CLOUD_PROJECT`	☑	GCP project ID	`corp-stro-salesinventory-prod`
`GOOGLE_CLOUD_LOCATION`	☑	GCP region	`us-central1`
`VERTEX_PROJECT_ID`	☑	Vertex AI project ID	Same as GOOGLE_CLOUD_PROJECT
`VERTEX_LOCATION`	☑	Vertex AI location	`us-central1`
`VERTEX_ENGINE_ID`	☑	Reasoning Engine ID	`5375474415045705728`
`FIRESTORE_COLLECTION`		Firestore collection for sessions	`user_sessions`
`SESSION_TIMEOUT_HOURS`		Session expiration (hours)	`8`
`ISPILOT_API_KEY`		API key for local testing	(none)

Endpoints

GET /health

Health check endpoint (no authentication required).

Response:

```
{
  "status": "healthy",
  "timestamp": "2026-08-28T10:30:45.123456"
}
```

POST /chat

Chat with IsPilot agents.

Request:

```
{
  "user_id": "sebastian",
  "message": "How is Talca Colin performing?",
```

```
  "session_id": "optional-session-uuid"
}
```

Response:

```
{
  "answer": "Response from IsPilot...",
  "session_id": "session-uuid",
  "request_id": "request-uuid",
  "timestamp": "2026-08-28T10:30:45.123456",
  "status": "ok"
}
```

Error Response:

```
{
  "error_code": "INVALID_API_KEY",
  "error_message": "Authentication failed",
  "request_id": "request-uuid"
}
```

Session Management

Automatic Session Handling

1. First request creates a session automatically
2. Subsequent requests reuse the session if valid
3. Sessions expire after `SESSION_TIMEOUT_HOURS` (default: 8 hours)
4. Session data persists in Firestore (production) or in-memory (development)

Explicit Session Control

```
# Reuse a specific session
curl -X POST http://localhost:8080/chat \
  -H "X-API-Key: your-api-key" \
  -d '{
    "user_id": "sebastian",
    "message": "Continue from previous context",
    "session_id": "8324292617089056768"
  }'
```

Session Storage

Production (Cloud Run):

- Primary: Cloud Firestore (`user_sessions` collection)
- Fallback: In-memory (if Firestore unavailable)
- Persists across service restarts
- Enables multi-instance deployments

Development (Local):

- In-memory session store
- Sessions reset on application restart
- Sufficient for testing and development

Deployment

To Cloud Run

```
chmod +x deploy.sh
./deploy.sh
```

The deployment script:

1. Builds Docker image (multi-stage, optimized)
2. Pushes to Artifact Registry
3. Deploys to Cloud Run with Workload Identity
4. Sets all required environment variables
5. Configures health checks and resource limits

Current Production Deployment:

- URL: `https://ispilot-api-46y2f3tyja-uc.a.run.app`
- Service Account: `sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com`
- Region: `us-central1`
- Status: ☑ Active

Authentication in Cloud Run

The deployment uses **Workload Identity**:

- No credential files stored in the container
- Service account automatically bound to Cloud Run service
- `google.auth.default()` automatically detects and uses Workload Identity
- No `GOOGLE_APPLICATION_CREDENTIALS` environment variable needed

This approach is secure and follows Google Cloud best practices.

Troubleshooting

Common Issues

Authentication Failed

- Verify Bearer token is valid: `gcloud auth print-identity-token --audiences <api-url>`
- For API Key auth: check `ISPILOT_API_KEY` environment variable
- Ensure service account has required IAM roles

Session Not Persisting

- Check Firestore is enabled: `gcloud services enable firestore.googleapis.com --project corp-stro-salesinventory-prod`
- Service account needs `roles/datastore.user` permission
- In-memory fallback is active if Firestore unavailable

Vertex AI Connection Failed

- Verify Engine ID is correct: `5375474415045705728`
- Check service account has `roles/aiplatform.user` permission
- Confirm region is `us-central1`

Technical Documentation

For implementation details, see:

- Deployment Guide - Setup and prerequisites
- Permissions & IAM - Service account setup and roles
- Session Service & Firestore - Session management and fallback
- Authentication Mechanism - Auth evolution and implementation
- Sprint Tracking - Checkpoint for resuming work

See Cloud Build Configuration below for details.

Configuration

Environment Variables

All environment variables are documented in `.env.example`:

```
# GCP Configuration
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json
GOOGLE_CLOUD_PROJECT=corp-stro-salesinventory-prod
GOOGLE_CLOUD_LOCATION=us-central1

# Vertex AI Configuration
VERTEX_PROJECT_ID=corp-stro-salesinventory-prod
VERTEX_LOCATION=us-central1
VERTEX_ENGINE_ID=5375474415045705728

# IsPilot API Configuration
FIRESTORE_COLLECTION=user_sessions
```

```
SESSION_TIMEOUT_HOURS=8
ISPILOT_API_KEY=your-api-key
```

Secrets Management

The API key is stored securely in Google Cloud Secret Manager:

```
# Create the secret
echo -n "your-secure-api-key" | gcloud secrets create ispilot-api-key \
  --project=corp-stro-salesinventory-prod \
  --replication-policy="automatic" \
  --data-file=-

# Grant access to service account
gcloud secrets add-iam-policy-binding ispilot-api-key \
  --project=corp-stro-salesinventory-prod \
  --member=serviceAccount:sa-ispilot-api@corp-stro-salesinventory-
prod.iam.gserviceaccount.com \
  --role=roles/secretmanager.secretAccessor
```

See `secrets.yaml` for complete secret configuration details.

Service Account

The service runs as `sa-ispilot-api@corp-stro-salesinventory-prod.iam.gserviceaccount.com` with required roles:

- `roles/aiplatform.user` - Access to Vertex AI Reasoning Engine
- `roles/datastore.user` - Access to Firestore sessions
- `roles/secretmanager.secretAccessor` - Access to API key secret
- `roles/logging.logWriter` - Write logs to Cloud Logging
- `roles/monitoring.metricWriter` - Write metrics to Cloud Monitoring

Cloud Build Configuration

The `cloudbuild.yaml` defines the CI/CD pipeline:

```
# Manually trigger a build
gcloud builds submit \
  --project=corp-stro-salesinventory-prod \
  --config=cloudbuild.yaml \
  --region=us-central1
```

Infrastructure as Code

The `deploy.yaml` Kubernetes manifest defines the Cloud Run service with:

- CPU/Memory: 2 CPU (limit) / 1 CPU (request), 1GB (limit) / 512MB (request)
- Concurrency: 100 requests per instance
- Timeout: 300 seconds (5 minutes)
- Health checks: Liveness (30s interval) and Readiness (5s interval)
- Environment variables and secrets (see Configuration section)

Local Development

Quick Start

```
chmod +x run.sh
./run.sh
```

The `run.sh` script:

1. Creates Python virtual environment
2. Installs dependencies from requirements.txt
3. Loads environment from .env
4. Starts uvicorn with auto-reload

Manual Setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/service-account.json"
export GOOGLE_CLOUD_PROJECT="corp-stro-salesinventory-prod"
export GOOGLE_CLOUD_LOCATION="us-central1"
export VERTEX_PROJECT_ID="corp-stro-salesinventory-prod"
export VERTEX_LOCATION="us-central1"
export VERTEX_ENGINE_ID="5375474415045705728"
export ISPILOT_API_KEY="your-test-api-key"
uvicorn app.main:app --reload --host 0.0.0.0 --port 8080
```

Testing

```
# Run all tests
pytest

# Run with coverage
pytest --cov=app tests/

# Run specific test file
pytest tests/test_health.py
```

```
# Run with verbose output
pytest -v
```

Troubleshooting

Authentication Issues

- **"Not authenticated with gcloud"**: Run `gcloud auth login`
- **"Permission denied" errors**: Verify service account has required IAM roles
- **"Secret not found"**: Ensure `ispilot-api-key` secret exists in Secret Manager

Health Check Failures

- Service may need time to start (first deployment can take 30-60 seconds)
- Verify Cloud Run resource limits are sufficient
- Check Cloud Run service logs: `gcloud run services logs read ispilot-api --region us-central1`

Cold Start Performance

- Multi-stage Docker build optimizes image size for faster cold starts
- Consider enabling Cloud Run's min instances if cold starts impact users

Architecture Notes

- The service hides all Vertex-specific request details behind `VertexAgentClient`
- Sessions are managed in Firestore with automatic expiration
- All requests include structured JSON logging with request IDs
- API key validation is enforced via middleware
- Authentication failures are logged for security auditing
- Vertex API calls include automatic retry with exponential backoff